

Challenge Brief

Which 15 gateways should our field team visit next week — and what kind of engineer are you?

LPDG INNOVATION HUB — SELECTION CHALLENGE 2026
PREPARED BY TARUN GUPTA · LPDG
AUDIENCE CANDIDATES



The situation

LPDG runs a radio network that carries meter readings back to us. About 320 gateways are installed on rooftops, in basements and in plant rooms. Each one relays readings for somewhere between 40 and 900 meters.

When a gateway starts failing, the meters behind it stop being read. Nobody notices straight away, because nothing breaks loudly. Weeks later it turns up as a wrong bill, someone driving out to read a meter by hand, and an annoyed customer.

The operations team can send out **15 site visits a week**. Today they choose those 15 from a spreadsheet and gut feel. Your job is to do better than that.

The data is made up. It is built to behave like a real network we run for a utility customer.

What we are really looking for

We are not looking for one type of person. We need **data engineers, software developers, DevOps engineers, data scientists, machine learning engineers and MLOps engineers**. We would rather work out which one you are from what you build than from your CV.

So there are two parts.

Part 1 is the same for everyone. It is small, and it does not need any machine learning.

Part 2 is one area you pick and go deep in. Which one you pick is part of what we read.

Part 1 — the basics

Everyone does this. It either works or it does not. **If it does not work, we do not look at Part 2.**

Build something that takes the data up to a given Monday and tells us **which 15 gateways to visit that week, in order, with a reason for each one**.

1. **It runs with one command** on a computer that is not yours. `docker compose up`, `make run`, or whatever you prefer, reading the data from a folder called `data`.
2. **It writes a file called** `predictions.csv` — 15 gateways for each of the 8 weeks from 2 February to 23 March 2026. 120 rows. Five columns: `week_start`, `rank`, `gateway_id`, `score`, `reason`. Check it with `validate_submission.py` before you send it.
3. `DECISIONS.md` — five choices you made, and for each one, what else you could have done and why you did not. One of the five must say **which area you picked in Part 2, and why**.
4. **What it cannot do** — where your thing falls over, and what another two weeks would fix.
5. `AI-USAGE.md` — what you used AI tools for, and one thing they got wrong that you spotted.
6. **A normal commit history**. Not one big commit at the end.
7. **A 6 to 8 minute screen recording** showing what you built and one result.

You do not need to build a model

We give you `baseline_3sigma.py`. It works, and it is close to what we actually run today. You are welcome to use it exactly as it is and never train anything.

If your strength is DevOps or writing software, do that. Spend your time on Part 2 instead. **Nobody loses marks for not doing machine learning.**

What mistakes cost

What happens	Cost
You send someone out	€380

What happens	Cost
They get there and nothing is wrong	€380, wasted
A broken gateway is left alone for a week	€600

The €600 happens again every week the gateway stays broken. The €380 happens once.

Fifteen visits is a hard limit. Not a goal, not a minimum.

We will run your `predictions.csv` through our scoring script and compare it to the baseline we gave you. **That number only checks that Part 1 works. It is not what we rank you on.** If you picked DevOps and your numbers match the baseline exactly, that is fine.

Things we have not told you

We have not said what "needs a visit" means. We have not said how early is early enough, how many false alarms are acceptable, or which measure matters most.

Those are yours to decide and yours to defend. If you think the task is not fully explained, you are right. Telling us that, with a view of your own, is worth more than a good number.

Part 2 — pick one area

Pick **one**. If you have spare time and want to do a second, say so and we will read it. But one done well beats two done badly, and the second will not make up for a weak first.

A — Data engineering

Build a proper pipeline, not a script that happened to work once.

- Raw data in, cleaned data out, in clear stages you can re-run.
- Safe to run twice. Running it again must not duplicate or double-count anything.
- Only processes what is new. Re-running after fresh data arrives should not redo everything.
- Checks that **stop and complain** when the data is wrong, instead of quietly giving a wrong answer.
- Write down what you expect the data to look like, and say what happens when it does not.
- A short note on what you found wrong in this data and what you did about it.

This data will give you trouble. That is the point of this area.

B — Software development

Wrap it in something another developer could pick up and work on.

- A web API: ask it for this week's 15; ask it why a particular gateway is where it is; tell it to run again.
- Real tests. Small ones, and at least one that runs the whole thing end to end. Include one test you wrote because you found a bug.
- Tidy structure. Someone should be able to swap out how the ranking works without touching the API.
- Handle things going wrong on purpose: bad input, missing data, a gateway that does not exist.
- Enough documentation that someone could call your API without asking you.

C — DevOps

Make it something a person who did not build it can run.

- In a container. One command to start. No manual steps.
- Automatic checks on every push: does it build, do the tests pass.

- Settings come from the environment, not from editing files. No paths that only exist on your laptop.
- A health check that would actually notice if the thing were broken.
- Logs worth reading, and a short page on how to run it, how to tell it is working, and what to do when it is not.

D — Data science

Answer the question we refused to answer.

- Decide what "needs a visit" means. Write it down. Say why, and say what other definition you considered and rejected.
- Test it honestly, and say what your test cannot tell you.
- Give a range, not one number. Say how much your result moves depending on which gateways you happened to test on.
- Turn the €380 and €600 into an actual decision. Where do you draw the line, and what does moving it cost in each direction?
- A write-up with charts, aimed at the operations manager rather than at us.

E — Machine learning

Beat the baseline, and know why you did.

- A model that beats `baseline_3sigma.py` on total cost. Not on accuracy.
- What you fed it, what you left out, and what it is actually paying attention to.
- A test someone trying to catch you out would still accept. In particular: test it on gateways it has never seen, and on weeks that came after the ones you trained on.
- What happens to it when the network changes.

F — MLOps

Everything around the model rather than the model itself.

- Training and predicting are genuinely separate steps, and the trained model is saved as a file with a version on it.
- Same inputs and same version give the same answer.
- Something watching the incoming data that would notice if it changed shape.
- A written rule for when you would retrain, and how you would know retraining made things worse.
- A way to go back to the previous version. One you have actually tried.

How we mark it

Weight	What
Pass or fail	Part 1. It runs, <code>predictions.csv</code> is valid, everything on the list is there. A fail stops here
60 %	The area you picked , against the points listed for it
25 %	Judgement — what you questioned about the task and the data, and how you defended your choices
15 %	Explaining it — <code>DECISIONS.md</code> , the recording, and whether your reasons would make sense to the operations manager

Knowing why you might be wrong matters more than being right. Something that does worse than the baseline, with a clear account of why, beats something that does better with no idea why.

If we invite you to a live session

Some people will be invited to a live session. It has three parts and takes about 35 minutes.

1. **We hand you a month of data nobody has seen** and ask your thing to run on it while we watch.
2. **We ask you to make one change, live**, in the area you picked. Not a rewrite — one change, with us in the room.
3. **We ask what another week would buy you.**

Part 2 is the part worth preparing for. Roughly what it looks like in each area:

Your area	The kind of change we would ask for
Data engineering	Make your pipeline survive an input that is broken in a new way
Software development	Add something to your API that is not there yet
DevOps	Find out why the running thing is giving the wrong answer
Data science	Move your threshold, and tell us what it costs in each direction
Machine learning	Make your model better on the month it has not seen
MLOps	Go back to your previous model version, and show us it worked

So build for it. **Keep training separate from predicting** — anything that retrains while it is trying to answer will not finish in the room. And know your own code well enough to change it with people watching, which usually means fewer clever parts rather than more.

We are not testing whether you get it right first time. How you behave when a change does not work is more useful to us than whether it works.

What you get

File	What it is
data/telemetry/month=YYYY-MM/*.parquet	What each gateway reported, every hour, Aug 2025 to Mar 2026
data/telemetry_sample_2025-08.csv	The same thing for one month, as a plain CSV
data/gateway_master.csv	The list of gateways and where they are
data/field_visits.csv	Past site visits and what the engineer found
data/meter_read_success.csv	How many meters were read each week, per gateway
data/engineer_review_2026-02.xlsx	One engineer's opinion of some gateways, from 15 Feb 2026
baseline_3sigma.py	A working baseline you may build on
validate_submission.py	Checks your predictions.csv before you send it
Data Dictionary	Every file and every column explained

Read the Data Dictionary before you start.

Dates

All times IST.

Four dates are fixed. Everything else happens in the week shown, and we will confirm the exact time by email.

When	What
Mon 24 Aug	You get this. Week 1 starts
Wed 26 Aug, 18:00	Send questions, first round
Week 1	Open session, one hour, recorded. Time confirmed by email. Answers to everyone the same evening
Week 2	Optional check-in — push something that starts. Not marked, no feedback
Mon 7 Sep, 18:00	Send questions, second round
Week 3	Second set of answers. No questions answered after this
Wed 16 Sep, 23:59	Hand in

Plan for **40 to 60 hours** across the three weeks. Part 1 should not take most of it.

The week 2 check-in costs you nothing and we do not look at it. It is there so that a project which does not start gets found in week 2 instead of on the day you hand in.

We will come back to you after the deadline. We are not putting a date on that, and you should not read anything into how long it takes.

Handing in

Work in a **private GitHub repository** on your own account. Keep it private the whole time. Nobody else should be able to see your work, and you should not be able to see theirs.

When you hand in, make it public and send us the link. It has to be public by 23:59 IST on Wednesday 16 September. If we cannot open it, we cannot mark it, and we will not chase you.

Two things before you make it public. **Do not put the data in your repository** — it is 104 MB and it is not yours to share. And check nothing private is in your commit history, like a password or a key. Making a repository public makes its whole history public.

Where your code should look for the data

We will put the data in a folder called `data` in the top level of your project, exactly as you received it. Your code should use that by default, and should also accept a different location if we give it one, for example `--data /somewhere/else`. If you use docker compose, mount `./data` into the container rather than copying the data inside the image.

Questions

Send questions by the dates above. Every answer goes to everyone, so asking costs you nothing and helps the whole group.

Questions about **practical things** — dates, file formats, how to hand in, what a column means — get a straight and complete answer.

Questions about **what you should build** will usually get "that is your call, tell us what you chose and why." That is not us dodging. The choice is the test.